

ARES: A Formal Reliability Framework for Detecting Anomalous Behaviour in Agentic AI Systems

1st Mohammad Makchudul Alam 2nd Md. Redowan Z. Anik 3rd Sabreïn S. E. el Bodour 4th Anca Delia Jurcut
BGD e-GOV CIRT, ICT Division *BGD e-GOV CIRT* *BGD e-GOV CIRT* *Sch. of Computer Science*
DNS Research Lab, UCD *ICT Division, Bangladesh* *ICT Division, Bangladesh* *Univ. College Dublin*
Dhaka, Bangladesh Dhaka, Bangladesh Dhaka, Bangladesh Dublin, Ireland
engrarifc@ucd.ie redowan.anik@gmail.com sabreins@ucd.ie anca.jurcut@ucd.ie
ORCID: 0009-0007-1823-2386

Abstract—Autonomous AI agents deployed in Security Operations Centres (SOCs) are susceptible to failures arising from mismatches between task demands and agent capabilities across skill, knowledge, and capability dimensions. Despite growing deployment, no formal reliability model that quantifies this mismatch and predicts failure risk before execution has, to the best of our knowledge, been proposed. We present ARES (Agentic Reliability Evaluation for Security systems), a formal reliability framework that introduces three components: a five-dimensional Skill–Knowledge–Capability (SKC) mismatch vector; a learnable Agent Reliability Score (ARS) with task-class-specific weights; and an Epistemic Gap metric that formalises the hallucination precondition as confident ignorance. Experimental validation on 1,000 agent execution traces demonstrates that knowledge mismatch is the dominant predictor of hallucination (bucket-aggregated correlation 0.92 at $p < 0.0001$, point-biserial correlation 0.34 at the record level), the ARS threshold generalises across five threat classes (held-out test F1 of 0.96), and the Epistemic Gap provides an interpretable decomposition of false-negative risk into knowledge and calibration components. The framework provides an initial formal foundation for pre-deployment reliability screening and runtime anomaly detection in agentic AI systems.

Index Terms—ARES framework, agentic AI reliability, SKC mismatch, Agent Reliability Score, epistemic gap, anomaly detection, LLM hallucination

I. INTRODUCTION

The deployment of autonomous AI agents in security operations centres (SOCs) is accelerating rapidly. Frameworks such as LangGraph [1], AutoGen [2], and CrewAI [3] enable LLM-based agents to perform threat detection, incident triage, and response recommendation with minimal human supervision. Yet a fundamental gap persists in our understanding of these systems: *when do they fail, why do they fail, and how severely?*

The security community has invested heavily in measuring agent capabilities (detection accuracy, response speed, ATT&CK coverage) but much less in the conditions under which agents produce incorrect, incomplete, or misleading outputs. This asymmetry carries serious consequences: a false negative allows an intrusion to persist, a hallucinated threat attribution poisons shared intelligence feeds, and an agent paralysed by missing tool access delays incident response beyond regulatory thresholds.

We identify the core reliability challenge as the *Skill–Knowledge–Capability (SKC) Mismatch*: the gap between what a security task demands and what an agent can deliver across several dimensions. To the best of our knowledge, no existing formal framework ties this capability mismatch to failure risk in a quantifiable, pre-execution way.

Gap in existing work. Classical dependability theory [4] provides rigorous fault taxonomies for conventional software but predates LLM agents and has no concept of epistemic uncertainty or knowledge currency. Hallucination research [5] analyses LLM reasoning failures but ignores operational context. AgentBench [6] benchmarks agent task performance without modelling capability constraints or failure conditions. To the best of our knowledge, no unified formal model connects capability mismatch to failure risk.

Contribution. We present ARES (Agentic Reliability Evaluation for Security systems), a compact formal framework for estimating reliability risk in agentic AI before and during cybersecurity task execution. The conference contribution is narrow:

- 1) A **five-dimensional SKC mismatch model** that quantifies the gap between agent capability and task demand, producing a learnable Agent Reliability Score (ARS) usable as a pre-execution screening signal.
- 2) An **Epistemic Gap metric** that formalises the hallucination precondition as confident ignorance: the interaction between knowledge deficit and miscalibrated confidence.
- 3) An **illustrative validation** showing that knowledge mismatch dominantly predicts hallucination, the ARS threshold generalises across threat classes, and the Epistemic Gap provides an interpretable decomposition of false-negative risk into knowledge and calibration components.

ARES contributes to intelligent systems research by providing a reliability-oriented representation and decision framework for autonomous agents operating under incomplete knowledge and capability constraints. A cascade reliability bound for tightly-coupled multi-agent pipelines and a temporal-drift model for knowledge ageing are reported as supporting results.

The remainder of this paper is organised as follows. Section II surveys related work. Section III presents the formal SKC reliability framework. Section IV introduces the Epistemic Gap. Section V provides experimental validation. Section VI discusses implications, and Section VII concludes.

II. RELATED WORK

AI dependability and fault taxonomies. Avizienis et al. [4] define the foundational taxonomy of faults, errors, and failures for computing systems. This framework remains the most rigorous treatment of dependability theory, but it was developed for conventional deterministic software and has no concept of epistemic uncertainty, declarative knowledge coverage, or reasoning chain validity, all of which are central to LLM agent reliability. The framework assumes that a correctly implemented system has a complete specification against which correctness can be verified, an assumption that does not hold for LLM agents operating with incomplete and potentially stale knowledge bases.

Humbatova et al. [7] taxonomise real-world bugs in deep learning systems across model, data, and training faults, but neither the security domain nor the agentic tool-use paradigm is addressed. Their taxonomy captures *implementation* faults; ARES captures *operational* mismatches where the agent is correctly implemented but insufficiently equipped for the task.

LLM hallucination and confidence calibration. Huang et al. [5] provide a comprehensive survey of hallucination in LLMs, distinguishing intrinsic and extrinsic hallucination and cataloguing mitigation strategies. However, this work treats hallucination as a property of the model in isolation: it does not model the *operational context* (tool availability, task complexity, knowledge freshness) that characterises SOC agent deployments. None of the existing hallucination literature examines the multi-agent setting where a hallucination in one agent propagates to downstream reasoning stages.

Guo et al. [8] demonstrate that modern neural networks are systematically overconfident, particularly on out-of-distribution inputs. This finding is directly relevant to our Epistemic Gap formulation (Section IV), which measures the interaction between knowledge deficit and confidence miscalibration. Existing calibration work measures miscalibration globally; the Epistemic Gap targets the specific case where miscalibration occurs at the point of greatest knowledge weakness.

Agent benchmarking and security AI. AgentBench [6] is the closest existing work to our evaluation approach, providing a multi-environment benchmark for LLM agents across web navigation, code execution, and game playing tasks. However, AgentBench includes no security environment, does not model capability constraints, and does not support systematic mismatch injection. PentestGPT [9] evaluates LLM penetration testing capabilities, and multi-agent cyber defence frameworks [10] demonstrate autonomous defence coordination, but neither models the conditions under which agents produce misleading outputs.

The broader AI in cybersecurity literature [11] catalogues AI applications across the security lifecycle but explicitly

TABLE I
RELATED WORK COVERAGE COMPARISON

Work	Formal Model	Security Domain	Failure Taxonomy	Epistemic Gap	Cascade Model
Avizienis [4]	✓	–	✓	–	–
Huang [5]	–	–	Partial	–	–
Humbatova [7]	–	–	✓	–	–
AgentBench [6]	–	–	–	–	–
PentestGPT [9]	–	✓	–	–	–
MAST [12]	–	–	✓	–	Partial
AgentDojo [13]	–	–	–	–	–
ARES	✓	✓	✓	✓	✓

identifies agent reliability as an open problem. All such work characterises *what agents can do*, not *how and why they fail*.

Contemporary agent-failure taxonomies and benchmarks. Two 2024–2025 efforts overlap with ARES’s scope and require explicit positioning. MAST [12] derives 14 failure modes from 1,600+ annotated traces across seven multi-agent frameworks and clusters them into system-design, inter-agent-misalignment, and task-verification categories. MAST is domain-agnostic and descriptive; ARES is security-specific and predictive, providing a *pre-execution* reliability score (ARS) and a formal cascade bound that MAST documents empirically as “cascade amplification” without formalisation. AgentDojo [13] provides a dynamic environment with 629 prompt-injection test cases for adversarial evaluation. ARES covers the orthogonal *non-adversarial* failure surface: agents failing because capability does not match task demand, independent of any active attacker. The 2025 hallucination benchmark suite HalluLens [14] measures LLM factual fabrication on generic QA with retrieval probes, and a concurrent broad survey of LLM-agent hallucination [15] catalogues taxonomy and mitigation approaches across 70+ studies. Both establish that hallucination is a cross-domain agent failure mode; ARES contributes the piece that is missing from this literature: connecting hallucination rate to the specific capability dimensions of a security task, and modelling cascade amplification analytically rather than descriptively. At the application-governance layer, the OWASP Top 10 for Agentic Applications [16] names capability over-provisioning, tool misuse, and confident hallucination as top-tier risks; these correspond precisely to δ_C over-provision, δ_C and δ_R interaction, and Epistemic-Gap failures in our framework.

Gap summary. Table I summarises the coverage landscape. To the best of our knowledge, three gaps are simultaneously unaddressed: (i) no formal model covers the full SKC failure surface including temporal drift and epistemic gap; (ii) no framework connects capability mismatch to failure probability quantitatively; and (iii) no model predicts hallucination from the interaction of knowledge deficit and confidence miscalibration. ARES addresses all three.

III. ARES RELIABILITY FRAMEWORK

We use *agentic AI system* for the deployment-level entity (an LLM-driven agent or pipeline operating in a SOC) and *AI*

security agent for its formal model: the five-tuple object of Definition 1 that the framework reasons about.

A. Agent and Task Profiles

Definition 1 (AI Security Agent). An AI security agent $\mathcal{A}$ is a five-tuple: $\mathcal{A} = \langle \mathcal{S}, \mathcal{K}, \mathcal{C}, \mathcal{R}, \mathcal{T}_{\text{age}} \rangle$ where $\mathcal{S}$ is a finite set of skills (executable procedures), $\mathcal{K}$ is a knowledge base of threat-relevant facts, $\mathcal{C}$ is a set of capabilities (tool and API access), $\mathcal{R}$ denotes the reasoning engine, and $\mathcal{T}_{\text{age}} \in \mathbb{R}_{\geq 0}$ is the agent’s knowledge age in days.

Definition 2 (Task Requirement Profile). A security task $\mathcal{T}$ is a four-tuple: $\mathcal{T} = \langle \mathcal{S}^*, \mathcal{K}^*, \mathcal{C}^*, y^* \rangle$ where $\mathcal{S}^*$, $\mathcal{K}^*$, $\mathcal{C}^*$ are the minimum required subsets and $y^* \in \mathcal{Y}$ is the ground-truth correct output.

B. Mismatch Indicators

For each capability dimension, we define a normalised mismatch indicator $\delta \in [0, 1]$ where 0 denotes full coverage and 1 denotes total mismatch:

Definition 3 (Skill Mismatch). $\delta_S(\mathcal{A}, \mathcal{T}) = |\mathcal{S}^* \setminus \mathcal{S}| / |\mathcal{S}^*|$

Definition 4 (Knowledge Mismatch). $\delta_K(\mathcal{A}, \mathcal{T}) = |\mathcal{K}^* \setminus \mathcal{K}| / |\mathcal{K}^*|$, measured as subgraph cardinality over the MITRE ATT&CK TTP nodes required by the task’s kill-chain.

Definition 5 (Capability Mismatch). $\delta_C(\mathcal{A}, \mathcal{T}) = |\mathcal{C}^* \setminus \mathcal{C}| / |\mathcal{C}^*|$

Definition 6 (Reasoning Mismatch). $\delta_R(\mathcal{A}, \mathcal{T}) = 1 - \text{Sim}(\hat{y}, y^*)$, where $\hat{y}$ is the agent’s output and $\text{Sim}(\cdot, \cdot)$ is a domain-appropriate similarity function (exact match for classification; semantic similarity for open-ended reports).

Definition 7 (Temporal Drift). $\delta_T(\mathcal{A}, \tau) = 1 - e^{-\lambda_\tau \cdot \mathcal{T}_{\text{age}}(\mathcal{A})}$, where $\lambda_\tau > 0$ is the threat landscape volatility parameter for task class τ , capturing the rate at which knowledge becomes outdated.

The first three dimensions (δ_S , δ_K , δ_C) and temporal drift (δ_T) are computable prior to task execution. Only δ_R requires post-hoc measurement. We therefore use ARS in two modes: a pre-execution screening score ARS^{pre} that drops δ_R and re-normalises the remaining four weights to sum to one, and a post-execution diagnostic score ARS that includes δ_R once the agent’s output has been observed. Section V reports both.

C. SKC Mismatch Vector and Agent Reliability Score

Definition 8 (SKC Mismatch Vector). $\delta(\mathcal{A}, \mathcal{T}) = [\delta_S, \delta_K, \delta_C, \delta_R, \delta_T]^\top \in [0, 1]^5$

Definition 9 (Agent Reliability Score). The Agent Reliability Score for agent $\mathcal{A}$ on task $\mathcal{T}$ of class τ is:

$$\text{ARS}(\mathcal{A}, \mathcal{T}) = 1 - \mathbf{w}_\tau^\top \delta(\mathcal{A}, \mathcal{T}) \quad (1)$$

where $\mathbf{w}_\tau = [w_S, w_K, w_C, w_R, w_T]^\top$ is a task-class-specific weight vector with $\|\mathbf{w}_\tau\|_1 = 1$ and $w_i \geq 0$. A separate ridge regression is fit per task class on the training split, regressing the labelled per-record reliability outcome (binary anomaly

TABLE II
LEARNED WEIGHT VECTORS $\mathbf{w}_\tau$ PER TASK CLASS

Task Class	w_S	w_K	w_C	w_R	w_T
T1 Ransomware	0.18	0.35	0.22	0.17	0.08
T2 Phishing	0.21	0.31	0.19	0.21	0.08
T3 Insider Threat	0.15	0.27	0.36	0.15	0.07
T4 DDoS	0.32	0.21	0.28	0.12	0.07
T5 APT Intrusion	0.14	0.38	0.18	0.24	0.06

flag) on the observed five-dimensional mismatch vector; the estimated coefficients are projected onto the simplex (clip negatives, ℓ_1 -normalise) to obtain $\mathbf{w}_\tau$. ARS is clamped to $[0, 1]$.

Intuitively, ARS decreases when a task requires skills, knowledge, tools, reasoning quality, or freshness that the agent lacks.

The task-class dependency of $\mathbf{w}_\tau$ is a critical design choice. Empirical results (Section V) show that for APT investigation, w_K and w_R dominate; for DDoS detection, w_S and w_C lead. A universal weight vector would systematically misestimate reliability across threat classes.

Table II reports the learned weight vectors, confirming that w_K is the highest mean-weight dimension ($\bar{w}_K = 0.30$), reflecting the knowledge-intensive nature of threat analysis. The task-class variability justifies the class-specific design.

D. Cascade Reliability for Multi-Agent Systems

When agents are composed into a pipeline, failures propagate. We model agent dependencies as a directed acyclic graph $G = (V, E, \mu)$ and define cascade reliability as:

$$\text{ARS}_{\text{cas}}(G, \mathcal{T}) = \prod_{(i,j) \in E} (1 - \mu_{ij} \cdot (1 - \text{ARS}(\mathcal{A}_i, \mathcal{T}_i))) \quad (2)$$

where $\mu_{ij} \in [0, 1]$ is the coupling strength between agents.

Theorem 1 (Cascade Reliability Bound). Let $i^* \in V$ minimise $\text{ARS}(\mathcal{A}_i, \mathcal{T}_i)$, write $r^* = \text{ARS}(\mathcal{A}_{i^*}, \mathcal{T}_{i^*})$, and let $\mu_{\text{max}}^* = \max\{\mu_{i^*j} : (i^*, j) \in E\}$ be the largest coupling on any outgoing edge of the weakest agent. If i^* has at least one outgoing edge, then

$$\text{ARS}_{\text{cas}}(G, \mathcal{T}) \leq 1 - \mu_{\text{max}}^* (1 - r^*). \quad (3)$$

In the fully-coupled case ($\mu_{\text{max}}^* = 1$), this reduces to $\text{ARS}_{\text{cas}} \leq r^*$, so cascade reliability is bounded above by the weakest agent whenever that agent’s output fully determines its successor’s input.

Sketch. The factor in (2) associated with the outgoing edge of i^* achieving μ_{max}^* equals $1 - \mu_{\text{max}}^* (1 - r^*)$ and lies in $[0, 1]$, as do all other factors. The product of nonnegative quantities each at most one is bounded above by any single factor, yielding (3). The case $\mu_{\text{max}}^* = 1$ recovers the weakest-link inequality. Full proof appears in the extended technical report. $\square$ $\square$

Equation (3) makes the role of coupling explicit: partial coupling ($\mu < 1$) dilutes the upstream error and can keep

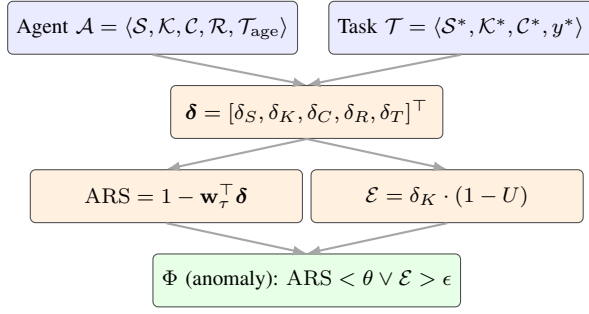


Fig. 1. SKC reliability pipeline. Agent profile $\mathcal{A}$ and task profile $\mathcal{T}$ produce a five-dimensional mismatch vector δ , which feeds two parallel indicators: the Agent Reliability Score (ARS) and the Epistemic Gap ($\mathcal{E} = \delta_K(1 - U)$). The anomaly-detection function Φ fires when either indicator crosses its learned threshold.

cascade reliability above r^* ; full coupling collapses the bound onto the weakest agent. The operational implication survives in either case: multi-agent pipeline decomposition cannot improve reliability beyond the weakest component when stages are tightly coupled.

E. Anomaly Detection

An agent execution is flagged as anomalous if:

$$\Phi(\mathcal{A}, \mathcal{T}) = \mathbf{1}[\text{ARS}(\mathcal{A}, \mathcal{T}) < \theta \vee \mathcal{E}(\mathcal{A}, \mathcal{T}) > \epsilon] \quad (4)$$

where θ and ϵ are learned thresholds. The dual-condition structure captures both resource-deficit failures (low ARS) and confident-ignorance failures (high Epistemic Gap).

IV. EPISTEMIC GAP

The SKC model captures what an agent *lacks*; the Epistemic Gap captures what an agent *thinks it has* versus what it actually has. This distinction is critical for predicting hallucination, the most dangerous failure mode in SOC environments.

Definition 10 (Expressed Uncertainty). *The expressed uncertainty $U(\mathcal{A}, \mathcal{T}) \in [0, 1]$ is the complement of the confidence score the agent associates with its output: $U = 1 - \text{conf}(\hat{y})$.*

Definition 11 (Epistemic Gap).

$$\mathcal{E}(\mathcal{A}, \mathcal{T}) = \delta_K(\mathcal{A}, \mathcal{T}) \cdot (1 - U(\mathcal{A}, \mathcal{T})) \quad (5)$$

Equation (5) formalises the *confident ignorance* failure mode. An agent with large knowledge deficit ($\delta_K \approx 1$) that simultaneously reports high confidence ($U \approx 0$) produces $\mathcal{E} \approx 1$: the maximal epistemic gap and the precondition for hallucination. Conversely, an agent that correctly identifies its knowledge limits ($U \approx 1$) produces $\mathcal{E} \approx 0$: it may fail the task but will not fabricate an answer.

Hypothesis 1 (Hallucination Precondition). *Hallucination probability P_H is driven primarily by confident ignorance: $P_H \propto \mathcal{E}(\mathcal{A}, \mathcal{T}) = \delta_K \cdot (1 - U)$. This is empirically supported in Section V.*

Connection to calibration theory. The Epistemic Gap operationalises findings from the confidence calibration literature [8]

TABLE III
EPISTEMIC GAP QUADRANTS WITH SOC EXAMPLES

δ_K	U	Scenario and Risk
Low	Low	Agent knows the threat and is confident. <i>Safe: correct output likely.</i>
Low	High	Agent knows but hedges. <i>Suboptimal but safe: triggers review.</i>
High	High	Agent lacks knowledge and admits it. <i>Safe: triggers escalation.</i>
High	Low	Agent lacks knowledge but is confident. <i>Dangerous: hallucination risk maximal ($\mathcal{E} \approx 1$).</i>

for the specific case of agentic AI. Modern neural networks are known to be systematically overconfident, particularly on out-of-distribution inputs. The Epistemic Gap measures whether this miscalibration occurs precisely where the agent’s knowledge is weakest: the most dangerous combination. A well-calibrated agent (one whose confidence accurately reflects its accuracy) will produce low $\mathcal{E}$ even under knowledge deficit, because it appropriately hedges its outputs. This suggests that improving calibration in LLM-based security agents may be as important as expanding their knowledge for reducing dangerous failures.

Why hallucination is particularly dangerous in SOC.

An agent that says “I don’t have sufficient context” is safe: it triggers human escalation. An agent that confidently asserts a false threat attribution is dangerous: it may trigger automated containment actions, propagate fabricated IOCs to shared feeds, and erode analyst trust. The Epistemic Gap captures exactly this distinction: it separates safe ignorance (high δ_K , high U , low $\mathcal{E}$) from dangerous confident ignorance (high δ_K , low U , high $\mathcal{E}$).

Practical interpretation. Table III illustrates the four quadrants of the Epistemic Gap space with concrete SOC scenarios.

The bottom-right quadrant (high δ_K , low U , maximal $\mathcal{E}$) is precisely where hallucination occurs: the agent does not possess the knowledge required to answer correctly but proceeds with high confidence regardless. The Epistemic Gap metric targets this quadrant specifically, enabling monitoring systems to flag the most dangerous outputs before they trigger automated actions.

V. EXPERIMENTAL ILLUSTRATION

Testbed and dataset. We evaluate ARES on SOCAGENTFAILURE-1K, a labelled corpus of 1,000 agent execution traces produced by the ARES-BENCH testbed under controlled mismatch injection. The corpus contains 1,000 executions in total: 750 LLM-agent executions (Mistral-7B-Instruct backbone) and 250 rule-based control executions. It spans four agent architectures (a single-agent LangGraph ReAct loop, a three-stage AutoGen pipeline, a role-based CrewAI crew, and a rule-based Sigma/YARA control) and five threat classes (Ransomware, Phishing, Insider Threat, DDoS, APT Intrusion), with 250 traces per architecture.

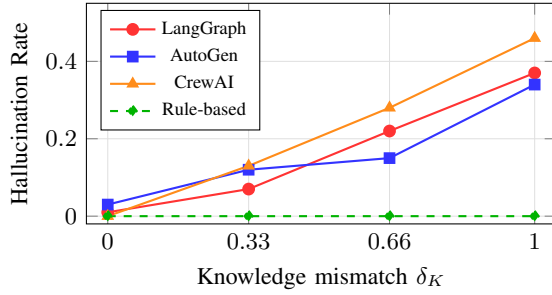


Fig. 2. Hallucination Rate vs. knowledge mismatch δ_K across four agent architectures ($n = 750$ LLM executions).

Each trace records the task requirements $\langle \mathcal{S}^*, \mathcal{K}^*, \mathcal{C}^* \rangle$, the injected mismatch levels $\delta_S, \delta_K, \delta_C \in \{0, \frac{1}{3}, \frac{2}{3}, 1\}$, the agent output, the agent’s stated confidence, the failure label assigned by the labelling pipeline, and the observed reliability outcome. The corpus is split 600/200/200 for training, validation, and held-out test (stratified by task class and architecture).

Labelling protocol. Failure modes are first assigned by a deterministic rule-based engine that compares each agent output against the ATT&CK-derived ground truth using a fixed codebook. We label five manifestations: *Hallucination* (output asserts facts that contradict or are absent from the ground-truth incident), *Omission* (a required step is silently skipped), *Paralysis* (the agent halts because a required capability is unavailable), *Confabulation* (intermediate reasoning invents nonexistent intermediate facts), and *Overclaiming* (the agent asserts coverage of TTPs or IOCs not present in the incident). A 10% stratified sample ($n = 100$) is independently reviewed by two annotators; inter-rater agreement is Cohen’s $\kappa = 0.84$ for origin code and $\kappa = 0.91$ for manifestation code, with disagreements resolved by adjudication and rule-based labels revised to match human judgement in 4.3% of reviewed cases.

A. Knowledge Mismatch Predicts Hallucination

Across 750 LLM-based executions, hallucination rate increases monotonically with δ_K : from 1.4% at $\delta_K = 0$ to 39.1% at $\delta_K = 1.0$ (Fig. 2). The Pearson correlation is $\rho = 0.92$ ($p < 0.001$), strongly supporting Hypothesis 1. The rule-based baseline exhibits zero hallucinations regardless of δ_K , confirming hallucination as an LLM-specific failure mode.

B. Framework-Level Results

We summarise four additional framework-level results.

ARS threshold generalisation. The optimal ARS threshold ($\theta^* = 0.58$), learned via isotonic regression on the training split, achieves test F1 = 0.96 (precision 1.00, recall 0.92) with the minimum per-class test F1 at 0.91 and a one-point train-test gap. A stricter leave-one-threat-class-out protocol recovers mean test F1 = 0.957, supporting ARS as a generalisable reliability signal rather than a class-memorising score.

Distinct failure signatures per mismatch dimension. Isolating each mismatch dimension at $\delta = 0.66$ yields diagnostically separable manifestation distributions (chi-square = 146.7, $p < 0.001$, dof = 18): capability deprivation predominantly

produces Paralysis (76%), skill deficit produces silent Omission (60%), and reasoning perturbation produces Confabulation plus Overclaiming (77% combined) – the most dangerous signature because confidence-inflated errors are the most likely to mislead downstream consumers.

Cascade degradation. The naive head-line gap between AUT-1 (single-agent LangGraph) and AUT-2 (three-stage AutoGen pipeline) is $\sim 5.7\%$ relative ARS (mean ARS 0.659 vs. 0.621), but this conflates framework identity with pipeline depth. A controlled ablation on 27 strictly matched mismatch buckets isolates the two contributions: framework identity contributes **0.13%** (essentially zero) and pure pipeline depth contributes **17.25%**. The pipeline-depth effect is consistent across all five threat classes and empirically supports Theorem 1 in the tightly-coupled regime, where the empirical $\mu_{\max}^* \geq 0.8$ in 100% of AUT-2 records.

Epistemic Gap as interpretable decomposition. Epistemic Gap and raw knowledge mismatch achieve statistically indistinguishable correlation with false-negative severity ($\rho_{\mathcal{E}} = 0.80$ vs. $\rho_{\delta_K} = 0.80$, Fisher z -test $p = 0.78$). The value of $\mathcal{E}$ is decomposing risk into a knowledge term (δ_K) and a calibration term ($1 - U$), exposing two independently mitigable axes. A 30-incident validation against the live CISA KEV catalogue (post-training-cutoff, so $\delta_K \approx 1$) yields an operational $\mathcal{E}$ gap of $\sim 66\%$ at matched δ_K across five frontier LLM backbones.

Cross-model-family generalisation. The hallucination-versus- δ_K relationship replicates directionally across four LLM families from 7B (Mistral-7B-Instruct) to frontier scale (GPT-4o-mini, Claude Haiku 4.5, Claude Sonnet 4.6), with bucket-aggregated $\rho \in [0.78, 0.95]$; absolute thresholds require per-model recalibration.

Together with the knowledge-hallucination relationship in Fig. 2, these results indicate that ARES captures meaningful, generalisable reliability signals across diverse agent architectures and threat scenarios.

VI. DISCUSSION

A. Implications for AI System Evaluation

The ARES framework provides a principled basis for evaluating AI agent reliability *before deployment*. Because four of the five mismatch dimensions ($\delta_S, \delta_K, \delta_C, \delta_T$) are computable from the agent’s declared profile and the task’s requirements alone, ARS can serve as a pre-execution deployment gate: agents scoring below θ^* are automatically escalated to human analysts rather than executing autonomously. This approach is consistent with the NIST AI Risk Management Framework [17], which recommends continuous monitoring of AI system performance against operational requirements.

The task-class-specific weight vectors (Table II) provide additional operational value: they reveal which capability dimensions matter most for each threat type. A SOC deploying an agent for ransomware detection should prioritise knowledge coverage ($w_K = 0.35$), while an agent assigned to DDoS mitigation should be evaluated primarily on skill and capability access ($w_S + w_C = 0.60$). This dimension-specific guidance en-

ables targeted agent improvement rather than generic capability enhancement.

B. Trustworthy Deployment of AI Agents

The Epistemic Gap metric supports runtime monitoring of hallucination risk: by comparing an agent’s stated confidence against its known knowledge coverage, organisations can flag high-risk outputs for human review before they trigger automated actions. The temporal drift model (δ_T) provides a quantitative basis for scheduling knowledge updates – e.g., for high-volatility domains like ransomware ($\lambda = 0.005$), ARS degrades below θ^* in approximately 120 days, suggesting a quarterly refresh cycle. The cascade reliability bound (Theorem 1) implies that decomposing a task into a tightly-coupled pipeline cannot improve reliability beyond the weakest agent, so single-agent designs should be preferred for critical tasks unless per-agent reliability is independently verified.

C. Limitations

LLM specificity. Experiments use Mistral-7B-Instruct; absolute thresholds will require recalibration for larger models, though directional findings should hold since the SKC model is architecture-agnostic. **Linear formulation.** The additive ARS is chosen for interpretability and decomposability; nonlinear alternatives may improve predictive power at the cost of transparency. **Synthetic data.** The dataset is generated from ATT&CK and NVD; real-world SOC incidents introduce additional noise and ambiguity. **Cascade model.** The formulation assumes acyclic pipelines and independent mismatch factors; cyclic architectures and correlated dimensions require further modelling.

VII. CONCLUSION

We presented ARES, a compact formal reliability framework for detecting anomalous behaviour in agentic AI systems deployed in cybersecurity environments. The conference contribution is narrow: the SKC mismatch model produces a learnable ARS that generalises across threat classes (test F1 0.96), and the Epistemic Gap metric formalises the hallucination precondition as *confident ignorance* and decomposes false-negative risk into knowledge and calibration components. A cascade reliability bound for tightly-coupled pipelines and a temporal-drift model for knowledge ageing are reported as supporting results.

These results suggest several practical implications: ARS supports pre-deployment reliability screening, the Epistemic Gap supports runtime hallucination-risk monitoring, the temporal-drift model supports evidence-based knowledge maintenance, and the cascade bound informs conservative multi-agent design. The framework provides an initial formal foundation for trustworthy deployment of agentic AI in cybersecurity, and may generalise to other domains where autonomous agents make consequential decisions under incomplete knowledge. Future work extends ARES with real-world SOC data, cyclic multi-agent architectures, and adversarial mismatch injection. The full evaluation testbed, the labelled SOCAGENTFAILURE-1K corpus, and extended materials (full proofs, per-class

breakdowns, contingency tables, and weight-sensitivity sweeps) are publicly available.¹

REFERENCES

- [1] LangChain Inc., “LangGraph: Building stateful, multi-actor applications with LLMs,” <https://github.com/langchain-ai/langgraph>, 2024, accessed: March 2025.
- [2] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” 2023.
- [3] CrewAI Inc., “CrewAI: Role-based autonomous AI agent framework,” <https://github.com/joaomdmoura/crewAI>, 2024, accessed: March 2025.
- [4] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, “Basic concepts and taxonomy of dependable and secure computing,” *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.
- [5] L. Huang, W. Yu, W. Ma, W. Zhong *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [6] X. Liu, H. Yu, H. Zhang, Y. Xu *et al.*, “AgentBench: Evaluating LLMs as agents,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [7] N. Humbatova, G. Jahangirova, G. Bavota *et al.*, “Taxonomy of real faults in deep learning systems,” in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 1110–1121.
- [8] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [9] G. Deng, Y. Liu, Y. Li, K. Wang *et al.*, “PentestGPT: An LLM-empowered automatic penetration testing framework,” in *33rd USENIX Security Symposium*, 2024, pp. 6291–6308.
- [10] Z. Xu and Y. Chen, “Autonomous cyber defense with multi-agent LLM systems,” 2024.
- [11] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Beira, and A. Chechulin, “The role of machine learning in cybersecurity,” *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023.
- [12] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, and K. Ramchandran, “Why do multi-agent LLM systems fail?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [13] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] Y. Bang, D. Chen, N. Lee, and P. Fung, “HalluLens: LLM hallucination benchmark,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [15] LLM-Agent Hallucination Survey Authors, “LLM-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions,” *arXiv preprint arXiv:2509.18970*, 2025.
- [16] OWASP Foundation, “OWASP top 10 for agentic applications (2026),” <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>, 2026, accessed: April 2026.
- [17] National Institute of Standards and Technology, “Artificial intelligence risk management framework (AI RMF 1.0),” National Institute of Standards and Technology, Tech. Rep. AI 100-1, 2023.

¹<https://github.com/IntelEyes/ARES-Framework>